AEGIS Backend MVP - Comprehensive Guide

Simple local backend with telemetry ingestion, normalized data, deterministic risk, and an audited AI stub.

API Service

FastAPI accepts Android telemetry, checks the enrollment token, validates schema v1, and returns 202 for accepted payloads.

Storage

SQLite is the local default. Raw payload JSON is kept on disk before processing for audit and replay.

Worker

Inline processing works locally. The same status fields can support a queue worker later.

Risk + AI

Rules produce the source of truth. The LLM analyzer is a structured, audited stub for high-risk cases.

What I made

- Created a new Python backend under backend/ using FastAPI, SQLAlchemy, pytest, and app settings.
- Copied the existing Android telemetry schema into the backend and validate every incoming payload against it.
- Implemented idempotent ingestion by payload_id, raw JSON storage, and local processing status tracking.
- Normalized device posture, current app inventory, and redacted important logs into queryable tables.
- Added deterministic risk scoring plus an LLMAnalyzer stub that only runs for suspicious/high-risk findings.
- Exposed the MVP API endpoints for latest risk, timelines, payload lookup, and analyst feedback.

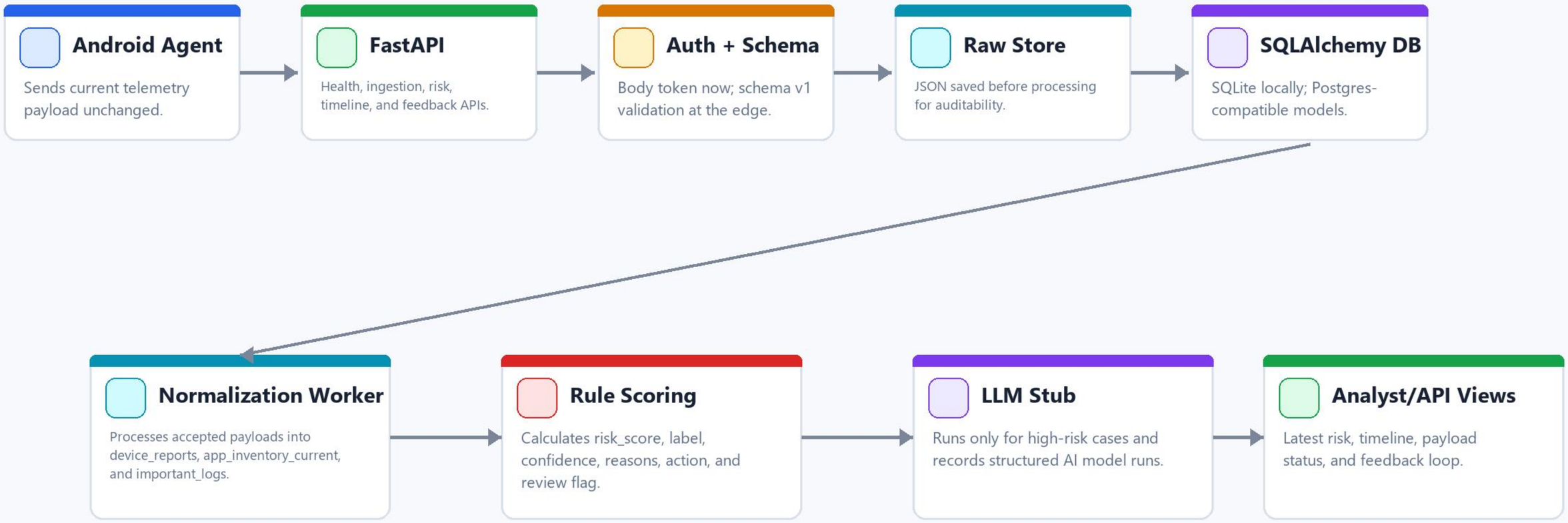
Verified: pytest 17 passed

Verified: /health OK

Verified: telemetry smoke processed

Runtime Architecture

The MVP keeps the backend small: API, database, raw store, local worker, rules, and AI audit trail.



AI boundary

- AI never validates the payload, mutates device state, or overrides deterministic storage decisions.
- AI receives a redacted evidence bundle and must return structured JSON with evidence references.
- Every model attempt is written to ai_model_runs for audit, debugging, and future provider comparison.

Data Model And Public API

The tables are intentionally boring and queryable; the API exposes only what the agent and analysts need now.

telemetry_payloads

payload_id, device_id, scan_id, raw_path, processing_status, processing_error, received_at

device_reports

device posture snapshot: root signals, integrity verdict, patch date, bootloader state

app_inventory_current

latest known app state by device and package, including source and permissions

important_logs

redacted log message, message hash, level, tag, matched rule

risk_assessments

risk_score, risk_label, confidence, reasons_json, recommended_action, review flag

ai_model_runs

provider, model, prompt/evidence hash, output_json, status, error

analyst_feedback

finding id, label, notes, analyst id, created_at for future evaluation

Public interfaces

- GET /health
- POST /api/v1/telemetry
- GET /api/v1/devices/{device_id}/latest-risk
- GET /api/v1/devices/{device_id}/timeline
- GET /api/v1/payloads/{payload_id}
- POST /api/v1/findings/{finding_id}/feedback

Run And Test Guide

Local commands for the MVP backend, plus the smoke test result from this run.

1. Run the API

cd C:\Users\ASUS\Desktop\mobile_sec\backend
python -m uvicorn app.main:app --host 127.0.0.1 --port 8080

Health URL:
http://127.0.0.1:8080/health

2. Run tests

cd C:\Users\ASUS\Desktop\mobile_sec\backend
python -m pytest

Current result:
17 passed

3. Optional worker

Inline processing is enabled locally.

For queue-style operation:
python -m app.worker --once
python -m app.worker

Smoke test performed

POST /api/v1/telemetry
accepted=true, duplicate=false, processing_status=PROCESSED

GET /api/v1/devices/sample-device-001/latest-risk
risk_score=100, risk_label=Critical, needs_human_review=true

GET /api/v1/payloads/manual-smoke-001
processing_status=PROCESSED and risk summary returned

Local defaults

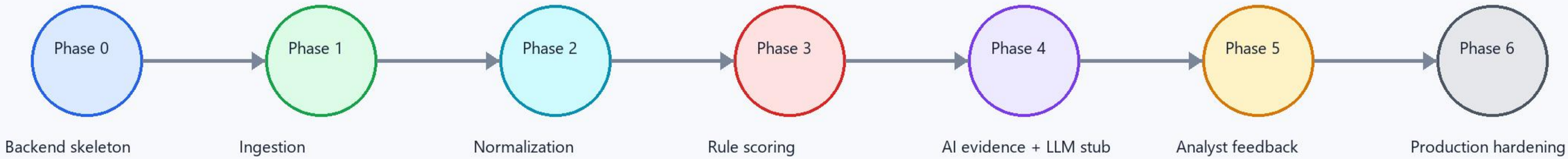
DB: backend-data/aegis.db

Raw JSON: backend-data/raw-payloads

Token: sample-token

AI-Aware Maturity Path

AI is important, but the first version keeps it constrained, testable, and easy to replace.



Now

Local MVP is implemented. It validates telemetry, stores raw payloads, normalizes data, scores risk, records AI stub runs, and accepts analyst feedback.

Next

Connect real Android uploads to this backend, add dashboard views, and use analyst feedback to tune rule thresholds before adding a real LLM provider.

Later

Docker Compose already sketches API, worker, Postgres, and Redis. Bearer auth, mTLS, Play Integrity backend decode, and real model providers can be swapped in behind existing interfaces.

- Keep deterministic rules as the authoritative decision layer.
- Use LLMs for explanation, evidence grouping, and analyst assistance after redaction.
- Evaluate future providers by comparing ai_model_runs against analyst_feedback labels.